Programación I

Teoría IV

http://proguno.unsl.edu.ar

proguno@unsl.edu.ar

DATOS ESTRUCTURADOS

Estructuras de Datos

- Hasta ahora hemos trabajado con …
  - Datos simples y arreglos
    - enteros
    - reales
    - Caracteres
    - Punteros
    - arreglos
- Sin embargo, existe la necesidad de manipular datos compuestos que no necesariamente sean del mismo tipo



REGISTROS

Estructuras de Datos Nueva

- Me pregunto nuevamente
  - ¿*Cuántos* elementos se pueden almacenar? (*Capacidad*)
  - ¿Cómo se *incorpora, elimina o cambia y se recupera* un elemento de la estructura? (*Operaciones*)
  - ¿En qué *orden* se encuentran elementos, uno con respecto a los otros? (*Orden)*
  - ¿Cómo se identifican o *seleccionan* los elementos de la estructura? (*Selector*).
  - ¿Qué *tipo de datos* pueden guardarse en la estructura? (*Tipo Base*).

Registros

- Llamados también tuplas, estructuras, records, structs, …
- Estructura heterogénea
- Selector: identificador de campo del registro (explícito)



- Capacidad estática.
- Operaciones: asignación e inspección.
- No hay orden en sus elementos.

Registros en C: Structs

```
struct nombre-registro {
   tipo nombre-campo;
   tipo nombre-campo;
   . . .
}
```

Los campos de un struct pueden ser de cualquier tipo simple o estructurado

```
struct per{
char apellido[20];
int edad;
float altura;
     }
```

```
struct fecha{
 int anio;
 int mes;
 int día;
          }
```

Registros en C: Structs

```
struct fecha{
   int anio;
   int mes;
   int dia;
} varFecha;
```

Declaración del tipo `struct fecha` y de la variable `varFecha`. Reserva memoria para la variable.

```
struct fecha fechaNacimiento,
             fechaCasamiento;
```

Declaración de las variables `fechaNacimiento` y `fechaCasamiento` de tipo `struct fecha`. Cada declaración reserva memoria.

Registros en C: Structs

- No es obligatorio darle un nombre al struct:

```
struct {
  int anio;
  int mes;
  int dia;
} varFecha;
```

  sin embargo, no lo vamos a poder usar en otra declaración de variable.

- Siempre debe estar presente al menos el nombre de la estructura o el nombre de la variable.

Operaciones con structs

- Es posible asignar un struct a otro, siempre que sean del mismo tipo, pero no se pueden comparar:

```
struct fecha{
   int anio;
   int mes;
   int dia;
} fechaNacimiento = { 1988, 10, 5};
struct fecha copiaFecha;
copiaFecha = fechaNacimiento;     /* OK */
if (copiaFecha == fechaNacimiento)
    printf("Iguales");                /* Error */
```

Operaciones con structs

- Obtener la dirección de un struct usando el operador de dirección (`&`).
- Acceder a sus campos por medio de su selector.
- Usar el operador `sizeof` para determinar su tamaño en bytes.
- Ser pasados como parámetros de una función y pueden también ser devueltos como resultado.

Inicialización de los structs

- Una variable de tipo struct también puede inicializarse al momento de la declaración

```
struct {
  int anio;
  int mes;
  int dia;
} fechaNacimiento = { 1988, 10, 5}
```

- Si faltan valores, inicializa en 0.
- Si no se hace al momento de la declaración, entonces hay que hacerlo campo a campo.

Acceso a los campos de un struct

```
struct {
  int anio;
  int mes;
  int dia;
} fechaNacimiento = {1988, 10, 5};

fechaNacimiento.dia = 29;
fechaNacimiento.mes = 6;
printf("La fecha de nacimiento de Juana es
  el %d/%d/%d\n", fechaNacimiento.dia,
  fechaNacimiento.mes,fechaNacimiento.anio);
```

Pasaje de un struct como parámetro de una función

- Al igual que cualquier otra variable en C, las variables de tipo struct son pasadas por valor.
- Para pasar un registro por dirección deberemos simular el pasaje por dirección (declarando el parámetro formal de tipo puntero y anteponiendo el operador de dirección & al parámetro actual).

Ejemplo de pasaje de un struct como parámetro de una función

```
#include <stdio.h>

/* declaracion global del tipo struct
  fecha*/
struct fecha{
  int anio;
  int mes;
  int dia;
};
```

```
void muestraFecha(struct fecha f){
 printf("%d/%d/%d\n", f.dia, f.mes, f.anio);
}

struct fecha modificaFecha(struct fecha f){
   f.dia = 29;
   f.mes = 6;
   f.anio = 2000;
   return f;
}

main(){
 struct fecha fechaNacimiento = { 1988, 10, 5};

 muestraFecha(fechaNacimiento);
 fechaNacimiento = modificaFecha(fechaNacimiento);
 muestraFecha(fechaNacimiento);
 return 0;
}
```

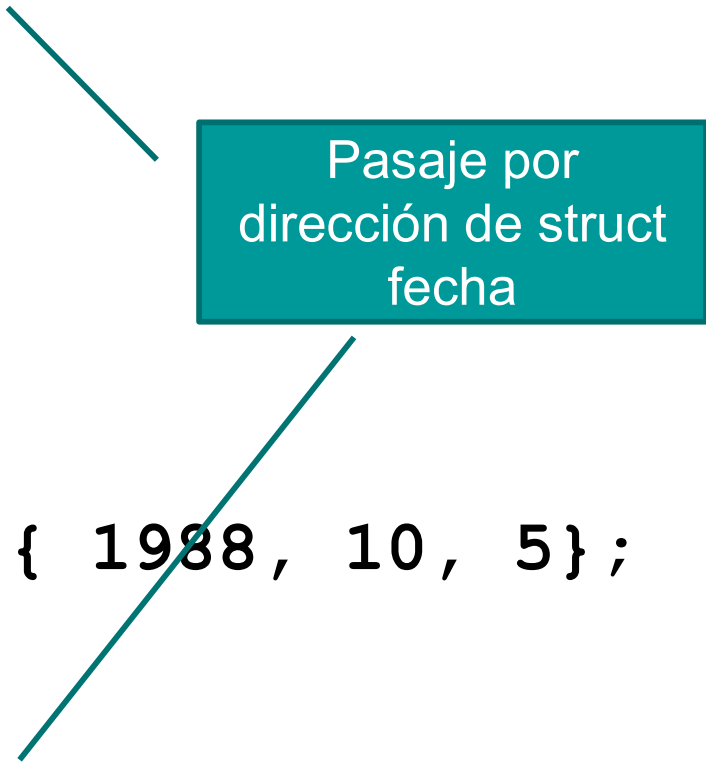
Pasajes por valor del struct fecha

```
void muestraFecha(struct fecha f) {
  printf("%d/%d/%d\n", f.dia, f.mes, f.anio);
}

void modificaFecha(struct fecha *f) {
   (*f).dia = 29;
   (*f).mes = 6;
   (*f).anio = 2000;
}

main(){
   struct fecha fechaNacimiento = { 1988, 10, 5};

   muestraFecha(fechaNacimiento);
   modificaFecha(&fechaNacimiento);
   muestraFecha(fechaNacimiento);
   return 0;
}
```



Uso de typedef

- **Permite crear un tipo**

```
typedef struct {
  int anio;
  int mes;
  int dia;
} Fecha;//define tipo llamado Fecha
void muestraFecha(Fecha f);
void modificaFecha(Fecha *f);
Fecha fecNac;//declara variable
```

Uso de typedef

```
typedef struct per{
char apellido[20];
int edad;
float altura;
     }Persona; //define tipo llamado Persona

void cargapersona(Persona *p);//prototipo
void muestrapersona(Persona p); //prototipo

Persona per1;//declara variable tipo persona
```

Ejemplos de carga

```
Persona pers1;//declara variable tipo persona
```

```
printf(“Ingrese apellido”);
scanf(“%s”,pers1.apellido);
printf(“Ingrese edad”);
scanf(“%d”,&pers1.edad);
printf(“Ingrese altura”);
scanf(“%f”,&pers1.altura);
```

Uso de `typedef`

- Permite crear sinónimos o alias para tipos de datos ya definidos:

```
struct fecha{
   int anio;
   int mes;
   int dia;
};
typedef struct fecha Fecha ;

void muestraFecha(Fecha f);
void modificaFecha(Fecha *f);

Fecha fecNac;
```

Punteros a structs

```
typedef struct triangulo {
  float base;
  float altura;
} Triangulo;
Triangulo t = {5.0, 10.0};
Triangulo *pTriang;
pTriang = &t;
(*pTriang).base = 6.0;
(*pTriang).altura = 5.8;
pTriang->base = pTriang->base / 2;
```

Arreglos de registros

- Es posible combinar ambos tipos de datos estructurados, así como dentro de un registro declaro un arreglo; un arreglo puede contener registros.
- Ejemplos sea

```
typedef struct per{
char apellido[20]; //string
int edad;
float altura;
      }Persona; //define tipo llamado Persona
```

Gráficamente `Persona gente[100];`

Lopez	Jofre	Perez	Funez	Garcia		
70	30	50	22	18	.....	
1,69	1,78	2,10	1,90	1,51		

0 1 2 3 4 99

```
printf("%s", gente[0].apellido);
strcpy(gente[5].apellido,"Lucero");
gente[5].edad=71;
gente [5].altura=1,98;
```

Arreglos de registros

```
Persona gente[100]; // arreglo de Persona
//ejemplo carga
for(int i=0; i<100;i++){
printf(“Ingrese apellido”);
scanf(“%s”,gente[i].apellido);
printf(“Ingrese edad”);
scanf(“%d”,&gente[i].edad);
printf(“Ingrese altura”);
scanf(“%f”,&gente[i].altura);
}
```

¿Y los punteros?

- `Persona gente[100];`
- `Persona *p;`
- `p=gente;`

```
printf(“%s”,(*p).nombre); ??
(*p).edad=30;  ??
```

¿Pensemos?

- ¿El arreglo siempre está lleno?
- ¿Es posible que necesitemos usarlo de a poco?
- ¿Cómo hago para controlar cuantos tengo en el arreglo?
- ¿El arreglo se sobreescribe?

